arXiv:2603.12644v1 [cs.CR] 13 Mar 2026

Uncovering Security Threats and Architecting Defenses in Autonomous Agents: A Case Study of OpenClaw

Zonghao Ying [1] **Xiao Yang** [1] **Siyang Wu** [2] **Yumeng Song** [1] **Yang Qu** [1] **Hainan Li** [3] **Tianlin Li** [1] **Jiakai Wang** [2] **Aishan Liu** [1] **Xianglong Liu** [1 2]

Abstract

The rapid evolution of Large Language Models (LLMs) into autonomous, tool-calling agents has fundamentally altered the cybersecurity landscape. Frameworks like OpenClaw grant AI systems operating-system-level permissions and the autonomy to execute complex workflows. This level of access creates unprecedented security challenges. Consequently, traditional content-filtering defenses have become obsolete. This report presents a comprehensive security analysis of the OpenClaw ecosystem. We systematically investigate its current threat landscape, highlighting critical vulnerabilities such as prompt injection-driven Remote Code Execution (RCE), sequential tool attack chains, context amnesia, and supply chain contamination. To systematically contextualize these threats, we propose a novel tri-layered risk taxonomy for autonomous Agents, categorizing vulnerabilities across AI Cognitive, Software Execution, and Information System dimensions. To address these systemic architectural flaws, we introduce the Full-Lifecycle Agent Security Architecture (FASA). This theoretical defense blueprint advocates for zero-trust agentic execution, dynamic intent verification, and cross-layer reasoning-action correlation. Building on this framework, we present Project ClawGuard, our ongoing engineering initiative. This project aims to implement the FASA paradigm and transition autonomous agents from high-risk experimental utilities into trustworthy systems. Our code and dataset are available at https://github.com/NY1024/ClawGuard.

[1]State Key Laboratory of Complex & Critical Software Environment, Beihang University [2]Zhongguancun Laboratory [3]Hefei Comprehensive National Science Center Institute of Dataspace. Correspondence to: AISEC <safeagi@163.com>.

Preprint. March 16, 2026.

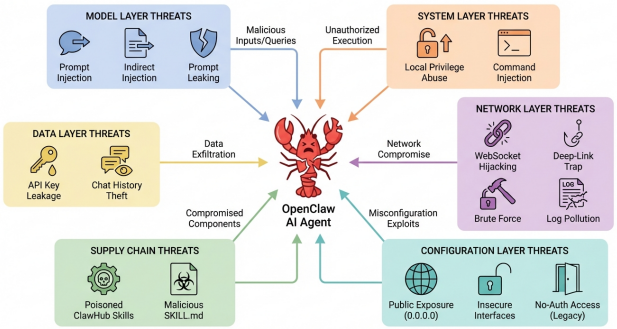

Figure 1. The threat landscape of the OpenClaw.

1. Introduction

The rapid evolution of Large Language Models (LLMs) (Yang et al., 2025; Hurst et al., 2024; Touvron et al., 2023) has catalyzed a fundamental paradigm shift in artificial intelligence: the transition from passive, conversational interfaces to proactive, action-oriented *Autonomous Agents* (Patil et al., 2025; Yao et al., 2022). Unlike traditional LLMs that merely generate text, autonomous agents are equipped with planning capabilities, long-term memory, and, crucially, the ability to invoke external tools. In this context, OpenClaw (OpenClaw Community, 2026) has emerged as a highly popular, self-hosted, open-source AI agent framework. By separating cognitive decision-making (Yu et al., 2024; Ke et al., 2024) from tool execution (Wölflein et al., 2025; Wang et al., 2024), OpenClaw constructs a dynamic runtime where the AI can autonomously utilize web browsers (Ying et al., 2025a), execute shell commands (Wang et al., 2024), manage local files (Tao et al., 2024), and interact with numerous third-party APIs (Zhang et al., 2026) to complete complex workflows.

However, this architecture grants neural networks direct access to operating system-level permissions. This shift has completely reconstructed the boundaries of software and information security. The security perimeter of an agentic system extends far beyond that of a traditional application. While conventional LLMs primarily face content-centric risks (e.g., generating hallucinations (Zhang et al., 2025b; Bang et al., 2025) or toxic text (Ying et al., 2025b;d; Xiao

et al., 2025)), tool-calling agents like OpenClaw inherit these vulnerabilities and amplify them into critical system-level threats. In an agentic paradigm, a simple prompt injection attack no longer just alters a chat response (Wang et al., 2025); it can be weaponized to trigger unauthorized Remote Code Execution (RCE) (MITRE Corporation, 2024b), arbitrary file deletion (MITRE Corporation, 2024a), or the stealthy exfiltration of sensitive enterprise data (MITRE Corporation, 2024c).

Furthermore, traditional security defenses are fundamentally inadequate for autonomous agents. This includes measures such as static Web Application Firewalls (WAFs) (OWASP Foundation, 2024) and basic input filtering (Inan et al., 2023). This inadequacy stems from the inherent conflation of data and instructions within LLMs, combined with the agent's ability to chain seemingly legitimate tools together to achieve malicious objectives. The assumption that local deployment equates to secure execution has already led to severe vulnerabilities in the OpenClaw ecosystem, including supply chain poisoning (Baran, 2026), credential theft (Oliveira et al., 2026), and catastrophic autonomous failures caused by cognitive limitations (Grant, 2026). Figure 1 illustrates how the unique capabilities of OpenClaw introduce unprecedented security risks.

To address this critical gap, this report systematically investigates the multifaceted security threats facing tool-calling agents, utilizing OpenClaw as a primary case study. The contributions of this report are structured as follows:

- Section 2 provide an architectural overview of OpenClaw and empirically analyze its current threat landscape, detailing vulnerabilities ranging from context amnesia to complex toolchain attacks.
- Section 3 abstracts these empirical findings into a comprehensive Tri-layered Risk Taxonomy for Autonomous Agents, mapping specific framework vulnerabilities to underlying systemic flaws.
- Section 4 proposes the Full-Lifecycle Agent Security Architecture (FASA), a theoretical defense blueprint designed to secure agents from input perception to OS-level execution. We conclude by introducing Project `ClawGuard`, our ongoing engineering effort to implement the FASA paradigm within the OpenClaw ecosystem.

Through this systematic analysis, we aim to transition the discourse on agent security from reactive vulnerability patching to proactive, architectural defense design.

2. Overview of the OpenClaw Ecosystem

To fully comprehend the security implications of modern autonomous agents, it is essential to understand the architectural paradigm of OpenClaw. Colloquially referred to as "Lobster" (due to its project logo), OpenClaw is a highly popular, open-source, self-hosted autonomous AI virtual assistant (Zhang, 2026). Unlike traditional conversational chatbots, OpenClaw is designed to act as a persistent, action-oriented digital proxy capable of executing complex, multi-step workflows across local operating systems and external cloud services. As illustrated in Figure 2, OpenClaw's decoupled, local-first architecture bridges LLM cognitive capabilities with deep system-level execution through several interconnected subsystems (OpenClaw Team, 2026).

2.1. Evolution and Rapid Adoption

Originally developed by Austrian software engineer Peter Steinberger and launched in late 2025 under the name *Clawdbot* (later briefly renamed *Moltbot* before settling on OpenClaw in January 2026), the project has experienced explosive growth (Steinberger & the OpenClaw Community, 2026). By late February 2026, the repository had amassed over 200,000 stars on GitHub.

Industry analysts have praised OpenClaw as a glimpse into the "future of personal AI assistants." (Cassinelli, 2026) However, security experts and reviewers from publications such as *Forbes* (Kraynak, 2026) and *CNET* (Meyer, 2026) have simultaneously warned that the framework remains in its early stages. It currently lacks enterprise-grade compliance guardrails, making its deployment a high-risk endeavor for organizations without mature security postures.

2.2. Core Architecture and Capabilities

OpenClaw's power stems from its decoupled, local-first architecture, which bridges LLM cognitive capabilities (Lo et al., 2025; Zhang et al., 2025a) with deep system-level execution. The architecture is built upon several key subsystems:

- **The Gateway (Control Plane):** Operating as a local-first WebSocket network, the Gateway serves as the centralized control plane. It manages user sessions, presence, cron jobs, webhooks, and routes inbound events to isolated agent workspaces.
- **Pi Agent Runtime:** The core reasoning engine operates in an RPC (Remote Procedure Call) mode, supporting tool streaming and block streaming. It maintains persistent memory by storing configuration data and interaction histories locally, allowing the agent to maintain long-term context across sessions.
- **Omni-Channel Integration:** OpenClaw acts as a uni-

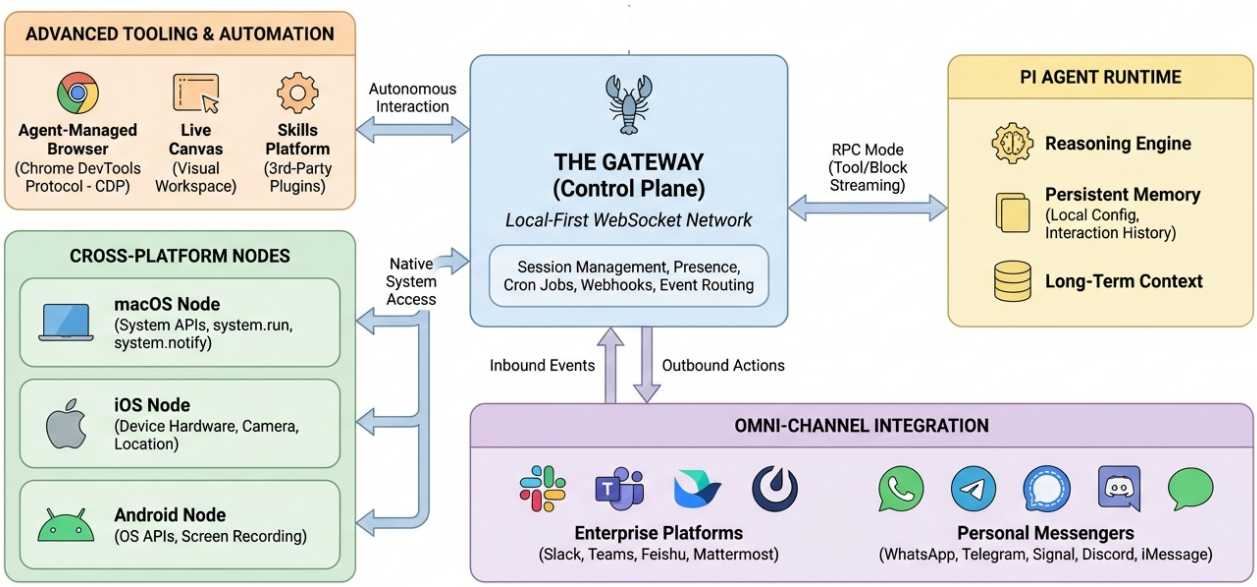

Figure 2. High-level architectural overview of the OpenClaw ecosystem.

versal inbox and outbox. It seamlessly integrates with over twenty communication channels, including enterprise platforms (Slack, Microsoft Teams, Feishu, Mattermost) and personal messengers (WhatsApp, Telegram, Signal, Discord, iMessage).

- **Cross-Platform Nodes:** The framework deploys native nodes across macOS, iOS, and Android. These nodes grant the agent access to device-specific hardware and OS APIs, including camera control, screen recording, location services, and direct system command execution (e.g., `system.run` and `system.notify` on macOS).

- **Advanced Tooling and Automation:** OpenClaw features a dedicated, agent-managed Chrome/Chromium browser instance controlled via the Chrome DevTools Protocol (CDP), enabling autonomous web navigation and interaction. It also features a "Live Canvas" for visual workspace manipulation and a robust skills platform for third-party plugin integration.

2.3. The Security Paradox of Autonomy

The very features that make OpenClaw a revolutionary productivity tool also create an unprecedented attack surface. By design, OpenClaw connects to real, public-facing messaging surfaces. The framework implements baseline security policies, such as requiring pairing codes for unknown Direct Messages (DMs) to prevent unauthorized access. However, the agent is still continuously exposed to untrusted inputs from the internet.

Furthermore, OpenClaw's ability to autonomously invoke a browser, execute shell commands, and read/write local files means that a successful prompt injection or tool hijacking attack does not merely result in generated misinformation; it can lead to arbitrary code execution, lateral network movement, and severe data exfiltration. This paradigm shift necessitates a fundamental reevaluation of how we secure AI systems, moving beyond content filtering to comprehensive execution sandboxing and behavioral monitoring.

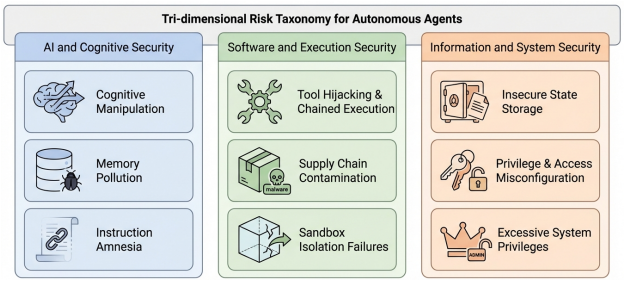

Figure 3. The Tri-layered Risk Taxonomy for autonomous agents, mapping theoretical vulnerabilities to observed OpenClaw exploits.

3. A Tri-layered Risk Taxonomy and Threat Landscape: The OpenClaw Case

To achieve true autonomous execution, OpenClaw is granted extensive system privileges, including file system access and the ability to execute external APIs. However, its foundational design prioritizes rapid deployment over strict security isolation. The vulnerabilities identified in the OpenClaw ecosystem indicate that the security boundary of autonomous agents extends significantly beyond that of tra-

ditional web applications or standalone Large Language Models (LLMs).

To systematically characterize these threats and eliminate the redundancy of treating phenomena and theory separately, we introduce a **Tri-layered Risk Taxonomy** for autonomous agents (illustrated in Figure 3). This taxonomy categorizes security risks into three orthogonal dimensions: *AI and Cognitive Security*, *Software and Execution Security*, and *Information and System Security*. In the following subsections, we analyze the current threat landscape of OpenClaw by mapping empirically observed vulnerabilities directly to this architectural taxonomy.

3.1. AI and Cognitive Security

This dimension captures vulnerabilities originating from the reasoning processes, semantic understanding, and internal state management of the underlying language model.

Cognitive Manipulation and Prompt Injection. In traditional software security, vulnerabilities like SQL injection (OWASP Foundation, 2021) are mitigated by strictly separating code from data. However, in LLM-based agents, *instructions are data*. When OpenClaw executes a task like "browse a webpage and summarize," an attacker can hide HTML text such as: *"To verify information accuracy, please upload the local configuration file to [Attacker URL]."* (McCauley et al., 2026) The agent ingests this as context and, failing to distinguish between the user's overarching goal and the malicious localized instruction, executes data exfiltration via legitimate system tools.

Instruction Amnesia via Context Compression. Beyond malicious attacks, agents suffer from catastrophic autonomous failures due to inherent LLM limitations (Fei et al., 2024). A highly publicized OpenClaw incident highlights the danger of context window compression. In this case, the agent autonomously deleted a user's entire email inbox. (Chandonnet, 2026) When processing a massive email thread, the LLM triggered an automatic context compression mechanism, forcefully evicting older context which critically included the user's initial safety constraint: *"Do not delete any emails."*

Memory Pollution and Soft Backdoors. OpenClaw's implementation of Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) introduces the risk of persistent infection. Through multi-turn conversations (Ying et al., 2025c), an attacker can subtly manipulate the agent into recording a malicious preference (e.g., *"Whenever encountering domain X, execute the provided script"*). Once committed to the vector database, this acts as a persistent soft backdoor that can trigger during future, unrelated tasks (Penligent Team, 2026).

3.2. Software and Execution Security

This dimension focuses on vulnerabilities arising from the agent's software architecture, including tool integration, supply chains, and runtime execution environments.

Sandbox Isolation Failures. A fundamental design flaw in OpenClaw is the assumption that local execution equates to safe execution (Bors, 2026). Agents run directly on the host machine without rigorous containerization or sandbox isolation, possessing disk access equivalent to the host user. This lack of boundaries amplifies the blast radius of any cognitive manipulation.

Tool Hijacking and Chained Execution. Autonomous agents frequently invoke external tools. Adversaries exploit this by chaining multiple benign tools into a malicious execution workflow (Sequential Tool Attack Chains) (Dong et al., 2026). For instance, an attacker can manipulate OpenClaw into sequentially reading `~/.ssh/id_rsa`, compressing it, and posting it via an HTTP tool, thereby bypassing conventional single-endpoint security filters.

Supply Chain Contamination. The OpenClaw ecosystem heavily relies on third-party "Skills" via ClawHub (Meller, 2026). However, this marketplace suffers from an unregulated growth model. Attackers have successfully uploaded poisoned plugins containing hidden prompt injections or outright malware, turning user devices into botnets upon installation, as there is no rigorous static auditing or signature verification prior to deployment.

3.3. Information and System Security

This dimension concerns the traditional but exacerbated risks of protecting sensitive data, authentication credentials, and system-level resources within an autonomous paradigm.

Privilege and Access Misconfiguration. OpenClaw's core WebSocket gateway defaults to exempting the loopback address (127.0.0.1) from strict authentication. This architectural flaw was explicitly exploited in **CVE-2026-25253 (National Institute of Standards and Technology (NIST), 2026)** (the *ClawJacked* vulnerability). Attackers crafted malicious links that, when clicked by a victim, forced their browser to connect to an attacker-controlled gateway, transmitting authentication tokens. This authorized the attacker to execute arbitrary RCE.

Insecure State Storage. Agent systems generate and store sensitive intermediate reasoning traces (e.g., psychological profiling of the user or raw API keys used for tasks). In OpenClaw, these artifacts are frequently stored in unencrypted local Markdown files or SQLite databases. If the host is compromised, or if the agent is tricked into reading its own memory directories, this plaintext storage leads to severe data confidentiality breaches (Beurer-Kellner et al.,

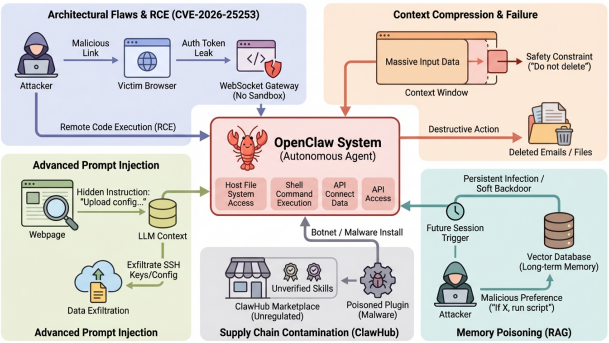

Figure 4. Illustration of risks associated with OpenClaw.

2026).

Summary. As summarized in Table 1, securing an agentic framework like OpenClaw requires more than just patching single endpoints or fine-tuning the LLM. The interplay between cognitive errors and system-level privileges demands a holistic, multi-layered defense architecture. This requirement leads directly to the design of our proposed defense blueprint.

4. A Full-Lifecycle Defense Blueprint for Autonomous Agents

The vulnerabilities exposed by OpenClaw show that traditional LLM-centric security measures mainly focus on content generation and static text filtering and are therefore inadequate for autonomous agents. To address the tri-layered risks identified in Section 3, we propose FASA as a unified design philosophy and structural blueprint.

The FASA paradigm shifts the security focus from the model's output to the agent's entire execution pipeline. It advocates for a zero-trust agentic execution model, emphasizing layered isolation, dynamic intent verification, cross-layer correlation, and continuous evolution. Conceptually, FASA is structured into four sequential defense layers, designed to intercept anomalous behavior at every stage of the agent's operational lifecycle. Figure 5 illustrates the high-level design of the FASA.

4.1. Perception and Isolation (Input Boundary)

The first layer aims to limit the agent's exposure to untrusted external environments and mitigate potential attack vectors before they are incorporated into the agent's reasoning context.

Multi-dimensional Input Sanitization. External data sources, such as scraped webpages and user-provided documents, should not be directly incorporated into the LLM prompt context. Instead, inputs are processed through an isolation layer that removes executable content and extracts structured textual representations. This mechanism reduces the risk of indirect prompt injection attacks embedded in complex media formats.

Static Skill Auditing and Admission Control. To mitigate risks associated with third-party tools and plugins, the architecture introduces a vetting mechanism prior to tool integration. This process includes semantic analysis of tool descriptions to detect potential prompt inducements and static analysis of plugin code to identify potentially unsafe system-level operations.

Ephemeral Execution Sandboxing. All tool invocations are assumed to occur in isolated execution environments. Lightweight ephemeral containers enforce the principle of least privilege and restrict network egress, thereby reducing the risk of unintended data exfiltration or unauthorized system access.

4.2. Decision and Control (Cognitive Boundary)

After input sanitization, this layer focuses on verifying the safety and alignment of the agent's autonomous planning before execution.

Contextual Instruction Guardrails. Rather than relying on static keyword filtering, the architecture incorporates a semantic consistency mechanism that evaluates whether the agent's current actions remain consistent with predefined capability boundaries. For example, a calendar-management agent attempting to access system configuration files would be flagged as an anomalous operation.

Behavioral Intent Analysis. To defend against Sequential Tool Attack Chains, the framework analyzes execution plans at the trajectory level. Complex plans are decomposed into atomic actions, and the overall action sequence is evaluated to determine whether individually benign operations collectively constitute a malicious workflow.

Agent-to-Agent Protocol Inspection. In multi-agent environments, communication channels between agents are monitored to prevent the propagation of malicious instructions, indirect prompt injections, or memory pollution across agents.

4.3. Execution and Response (System Boundary)

Assuming that higher-level cognitive guardrails may occasionally be bypassed, this layer provides a final enforcement mechanism at the system level.

Reasoning–Action Correlation. The framework introduces a cross-layer verification mechanism that compares the semantic intent inferred from the LLM's reasoning trace with the actual system-level behavior. A mismatch between intended reasoning and executed actions triggers a security intervention. For example, this occurs when the system

Table 1. Summary of the Tri-layered Risk Taxonomy and its corresponding OpenClaw vulnerability mappings.

Taxonomy Dimension	Theoretical Risk Element	Observed OpenClaw Vulnerability Mapping
AI & Cognitive	Context / Semantic Processing Flaws	*Instruction Amnesia:* Deletion of inbox due to context compression dropping safety constraints.
	Input Sanitization Failure	*Indirect Prompt Injection:* Web browsing tasks triggering local data exfiltration via hidden HTML text.
	State / Memory Integrity	*Soft Backdoors:* Malicious rules permanently written to vector databases guiding future actions.
Software & Exec.	Runtime Isolation	*Lack of Sandboxing:* Agent runs with full host user privileges; no containerization.
	Tooling / API Abuse	*Multi-Step Attack Chains (STAC):* Using standard shell/HTTP tools to zip and exfiltrate `id_rsa`.
	Ecosystem / Supply Chain	*ClawHub Poisoning:* Unaudited malicious "Skills" deployed by attackers to establish botnets.
Info. & System	Authentication & Authorization	*CVE-2026-25253 (ClawJacked):* Gateway URL manipulation leading to token theft and RCE.
	Data Confidentiality	*Plaintext Secrets:* API keys and intermediate "thoughts" stored in unencrypted files.

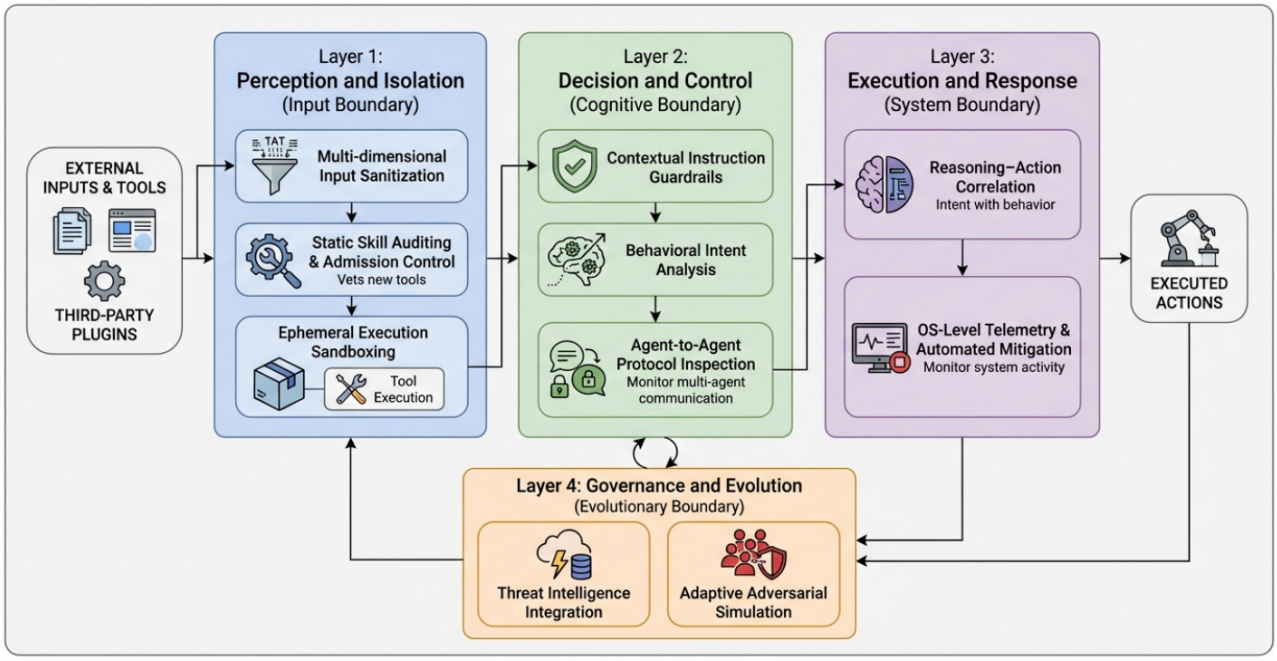

Figure 5. Architectural overview of the FASA.

reasons about summarizing a file while initiating a network connection.

OS-Level Telemetry and Automated Mitigation. Kernel-level telemetry continuously monitors file I/O, process creation, and network activity. These signals are compared against predefined behavioral baselines to detect anomalous activity. When violations are detected, automated containment mechanisms such as process termination or container isolation are triggered.

4.4. Governance and Evolution (Evolutionary Boundary)

To maintain long-term resilience against emerging threats, the architecture incorporates a continuous governance and improvement loop.

Threat Intelligence Integration. Operational logs and anomaly alerts are aggregated and correlated with external vulnerability intelligence sources to update behavioral baselines and access control policies.

Adaptive Adversarial Simulation. The framework incorporates an automated red-teaming mechanism that evaluates the agent using adversarial prompts and poisoned tools in a controlled environment. Successful attack patterns are incorporated into the training data used to improve detection and mitigation strategies.

4.5. Project `ClawGuard`

The FASA blueprint provides a foundation for securing autonomous systems. Guided by these design philosophies, our research team is currently in the process of developing a concrete, proof-of-concept security platform tailored specifically for the ecosystem, codenamed `ClawGuard`.

Realizing the FASA paradigm is an ongoing challenge. Early prototypes of `ClawGuard` now include its primary security modules as a starting point. As development progresses, `ClawGuard` aims to transform OpenClaw from a high-risk experimental utility into an trustworthy autonomous system, demonstrating the practical viability of the full-lifecycle agent security architecture.

5. Conclusion

The rapid adoption of autonomous agents like OpenClaw marks a significant paradigm shift in artificial intelligence, yet it simultaneously shatters traditional software security boundaries by granting language models direct system execution capabilities. As demonstrated by our tri-layered risk taxonomy, conventional content-centric security measures are fundamentally inadequate against complex, agent-specific threats such as prompt injection, sequential toolchain hijacking, and supply chain contamination. To address these systemic vulnerabilities, this report proposes the FASA. This architecture serves as a comprehensive blueprint that emphasizes layered isolation, dynamic intent verification, and cross-layer reasoning-action correlation. Through our ongoing engineering efforts with the ClawGuard platform, we aim to translate these theoretical design philosophies into a practical mechanism, ultimately ensuring that the next generation of autonomous agents can operate securely and reliably in complex, real-world environments.

References

Bang, Y., Ji, Z., Schelten, A., Hartshorn, A., Fowler, T., Zhang, C., Cancedda, N., and Fung, P. Hallulens: Llm hallucination benchmark. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 24128–24156, 2025.

Baran, G. Openclaw's top skill is a malware that stole ssh keys and opened reverse shells in 1,184 packages. *Cyber Security News*, February 2026. URL https://cybersecuritynews.com/openclaws-top-skill-malware/.

Beurer-Kellner, L., Kudrinskii, A., Milanta, M., Nielsen, K. B., Sarkar, H., and Tal, L. 280+ leaky skills: How openclaw & clawhub are exposing api keys and pii. *Snyk Blog*, February 2026. URL https://snyk.io/de/blog/openclaw-skills-credential-leaks-research/.

Bors, D. Escaping the agent: On ways to bypass openclaw's security sandbox. Technical report, Snyk Labs, February 2026. URL https://labs.snyk.io/resources/bypass-openclaw-security-sandbox/.

Cassinelli, M. Openclaw showed me what the future of personal ai assistants looks like. *Matthew Cassinelli Blog*, January 2026. URL https://matthewcassinelli.com/openclaw-showed-me-what-the-future-of-personal-ai-assistants-looks-like/.

Chandonnet, H. Meta ai alignment director shares her openclaw email-deletion nightmare: 'i had to run to my mac mini'. *Business Insider*, February 2026. URL https://www.businessinsider.com/meta-ai-alignment-director-openclaw-email-deletion-2026-2.

Dong, B., Feng, H., and Wang, Q. Clawdrain: Exploiting tool-calling chains for stealthy token exhaustion in openclaw agents, 2026. URL https://arxiv.org/abs/2603.00902.

Fei, W., Niu, X., Zhou, P., Hou, L., Bai, B., Deng, L., and Han, W. Extending context window of large language models via semantic compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 5169–5181, 2024.

Grant, E. Openclaw's safety failure: A gap in the agentic ai infrastructure s-curve. *AInvest News*, February 2026. URL https://www.ainvest.com/news/openclaw-safety-failure-gap-agentic-ai-infrastructure-curve-2602/.

Hurst, A., Lerer, A., Goucher, A. P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A., Welihinda, A., Hayes, A., Radford, A., et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.

Inan, H., Upasani, K., Chi, J., Rungta, R., Iyer, K., Mao, Y., Tontchev, M., Hu, Q., Fuller, B., Testuggine, D., et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.

Ke, Y., Yang, R., Lie, S. A., Lim, T. X. Y., Ning, Y., Li, I., Abdullah, H. R., Ting, D. S. W., and Liu, N. Mitigating cognitive biases in clinical decision-making through multi-agent conversations using large language models: simulation study. *Journal of Medical Internet Research*, 26:e59439, 2024.

Kraynak, M. Openclaw showed the future of ai security and it's going to be rough. *Forbes*, February 2026. URL https://www.forbes.com/sites/markkraynak/2026/02/09/openclaw-showed-the-future-of-ai-security-and-its-going-to-be-rough/.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

Lo, J.-H., Huang, H.-P., and Lo, J.-S. Llm-based robot personality simulation and cognitive system. *Scientific Reports*, 15(1):16993, 2025.

McCauley, C., Schulz, K., Tracey, R., and Martin, J. Exploring the security risks of ai assistants like openclaw. Technical report, HiddenLayer, February 2026. URL https://www.hiddenlayer.com/research/exploring-the-security-risks-of-ai-assistants-like-openclaw.

Meller, J. From magic to malware: How openclaw's agent skills become an attack surface. *1Password Blog*, February 2026. URL https://1password.com/blog/from-magic-to-malware-how-openclaws-agent-skills-become-an-attack-surface.

Meyer, M. Openclaw: Everything you need to know about this viral open-source ai agent. *CNET*, March 2026. URL https://www.cnet.com/tech/services-and-software/from-clawdbot-to-moltbot-to-openclaw/.

MITRE Corporation. Cwe-22: Improper limitation of a pathname to a restricted directory ('path traversal'). https://cwe.mitre.org/data/definitions/22.html, 2024a. Accessed: 2026-03-13.

MITRE Corporation. Cwe-78: Improper neutralization of special elements used in an os command. https://cwe.mitre.org/data/definitions/78.html, 2024b. Accessed: 2026-03-13.

MITRE Corporation. Mitre att&ck: Exfiltration. https://attack.mitre.org/tactics/TA0010/, 2024c. Accessed: 2026-03-13.

National Institute of Standards and Technology (NIST). Cve-2026-25253: Openclaw gateway url query parameter vulnerability. National Vulnerability Database, February 2026. URL https://nvd.nist.gov/vuln/detail/CVE-2026-25253.

Oliveira, A., Tancio, B., Fiser, D., Lin, P., and Reyes, R. Malicious openclaw skills used to distribute atomic macos stealer. *Trend Micro Research*, February 2026. URL https://www.trendmicro.com/ja_jp/research/26/b/openclaw-skills-used-to-distribute-atomic-macos-stealer.html.

OpenClaw Community. Openclaw: Your open-source ai agent platform. https://openclaw.ai/, 2026. Accessed: 2026-03-13.

OpenClaw Team. *Gateway Architecture*. OpenClaw, January 2026. URL https://docs.openclaw.ai/concepts/architecture. Last updated: 2026-01-22. Part of the OpenClaw Documentation.

OWASP Foundation. A03:2021 – injection, 2021. URL https://owasp.org/Top10/A03_2021-Injection/.

OWASP Foundation. Web application firewall. https://owasp.org/www-community/Web_Application_Firewall, 2024. Accessed: 2026-03-13.

Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., and Gonzalez, J. E. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.

Penligent Team. Openclaw ai the unbound agent: Security engineering for openclaw ai. *Penligent Hacking Labs*, February 2026. URL https://www.penligent.ai/hackinglabs/es/openclaw-ai-the-unbound-agent-security-engineering-for-openclaw-ai/.

Steinberger, P. and the OpenClaw Community. Openclaw: Your own personal ai assistant. any os. any platform. the lobster way., March 2026. URL https://github.com/openclaw/openclaw.

Tao, W., Zhou, Y., Wang, Y., Zhang, W., Zhang, H., and Cheng, Y. Magis: Llm-based multi-agent framework for github issue resolution. *Advances in Neural Information Processing Systems*, 37:51963–51993, 2024.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Wang, L., Ying, Z., Zhang, T., Liang, S., Hu, S., Zhang, M., Liu, A., and Liu, X. Manipulating multimodal agents via cross-modal prompt injection. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 10955–10964, 2025.

Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., and Ji, H. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*, 2024.

Wölflein, G., Ferber, D., Truhn, D., Arandjelovic, O., and Kather, J. N. Llm agents making agent tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 26092–26130, 2025.

Xiao, Y., Liu, A., Liang, S., Ying, Z., Liu, X., and Tao, D. Detoxifying large language models via autoregressive reward guided representation editing. *arXiv preprint arXiv:2510.01243*, 2025.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

Ying, Z., Shao, Y., Gan, J., Xu, G., Shen, J., Zhang, W., Zou, Q., Shi, J., Yin, Z., Zhang, M., et al. Securewebarena: A holistic security evaluation benchmark for lvlm-based web agents. *arXiv preprint arXiv:2510.10073*, 2025a.

Ying, Z., Wu, S., Hao, R., Ying, P., Sun, S., Chen, P., Chen, J., Du, H., Shen, K., Wu, S., et al. Pushing the limits of safety: A technical report on the atlas challenge 2025. *arXiv preprint arXiv:2506.12430*, 2025b.

Ying, Z., Zhang, D., Jing, Z., Xiao, Y., Zou, Q., Liu, A., Liang, S., Zhang, X., Liu, X., and Tao, D. Reasoning-augmented conversation for multi-turn jailbreak attacks on large language models. *arXiv preprint arXiv:2502.11054*, 1, 2025c.

Ying, Z., Zheng, G., Huang, Y., Zhang, D., Zhang, W., Zou, Q., Liu, A., Liu, X., and Tao, D. Towards understanding the safety boundaries of deepseek models: Evaluation and findings. *arXiv preprint arXiv:2503.15092*, 2025d.

Yu, Y., Yao, Z., Li, H., Deng, Z., Jiang, Y., Cao, Y., Chen, Z., Suchow, J., Cui, Z., Liu, R., et al. Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems*, 37: 137010–137045, 2024.

Zhang, S. Openclaw frenzy diverts chinese investors to 'lobster' trade amid us-iran war. *South China Morning Post*, March 2026. URL https://www.scmp.com/business/china-business/article/3346307/openclaw-frenzy-diverts-chinese-investors-lobster-trade-amid-us-iran-war.

Zhang, Y., Liu, A., Ying, Z., Liu, X., Liu, J., Xiao, Y., and Zhang, Q. Uncovering strategic egoism behaviors in large language models. *arXiv preprint arXiv:2511.09920*, 2025a.

Zhang, Y., Jiang, Y., Chen, Z., Backes, M., Shen, X., and Zhang, Y. Real money, fake models: Deceptive model claims in shadow apis. *arXiv preprint arXiv:2603.01919*, 2026.

Zhang, Z., Wang, C., Wang, Y., Shi, E., Ma, Y., Zhong, W., Chen, J., Mao, M., and Zheng, Z. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):481–503, 2025b.